or String. Instead, it gives the object passed in the opportunity to represent itself as a number or a string. If it were written in Ruby, it may look something like this:

```
class File
  def self.new(file, *args)
    if file.respond_to?(:to_int)
      IO.new(file.to_int, *args)
    else
      name = file.to_str
      # call operating system to open file 'name'
    end
  end
end
```

So, let's see what happens if we want to pass a file descriptor integer stored as a Roman number into new. Because our class implements to_int, the first respond_to? test will succeed. We'll pass an integer representation of our number to open, and the file descriptor will be returned, all wrapped up in a new IO object.

A small number of strict conversion functions are built into the standard library:

to_ary → Array
: Used when the interpreter needs a parameter to a method to be an array and when expanding parameters and assignments containing the *xyz syntax on the method definition side. In this case, puts does this conversion in its method definition.

```
class OneTwo
  def to_ary
    [1, 2]
  end
end

ot = OneTwo.new
puts ot
```

produces:

```
1
2
```

to_a → Array
: Used when the interpreter needs to convert an object into an array for parameter passing on the caller side of the method call or during parallel assignment.

```
class OneTwo
  def to_a
    [1, 2]
  end
end

ot = OneTwo.new
a, b = *ot
puts "a = #{a}, b = #{b}"
printf("%d -- %d\n", *ot)
```

produces:

```
a = 1, b = 2
1 -- 2
```